

# Introduction to reinforcement learning

Learning to make decisions from experience

# Brainstorm

- What is intelligence?

“Perceive, infer and retain information and apply it towards goals”

Learning is mostly here.

Planning and Reinforcement Learning (loosely)

- What is artificial intelligence?

“The above but for artificial agents”

A better alternative: “The science of making **agents** that **act rationally**”

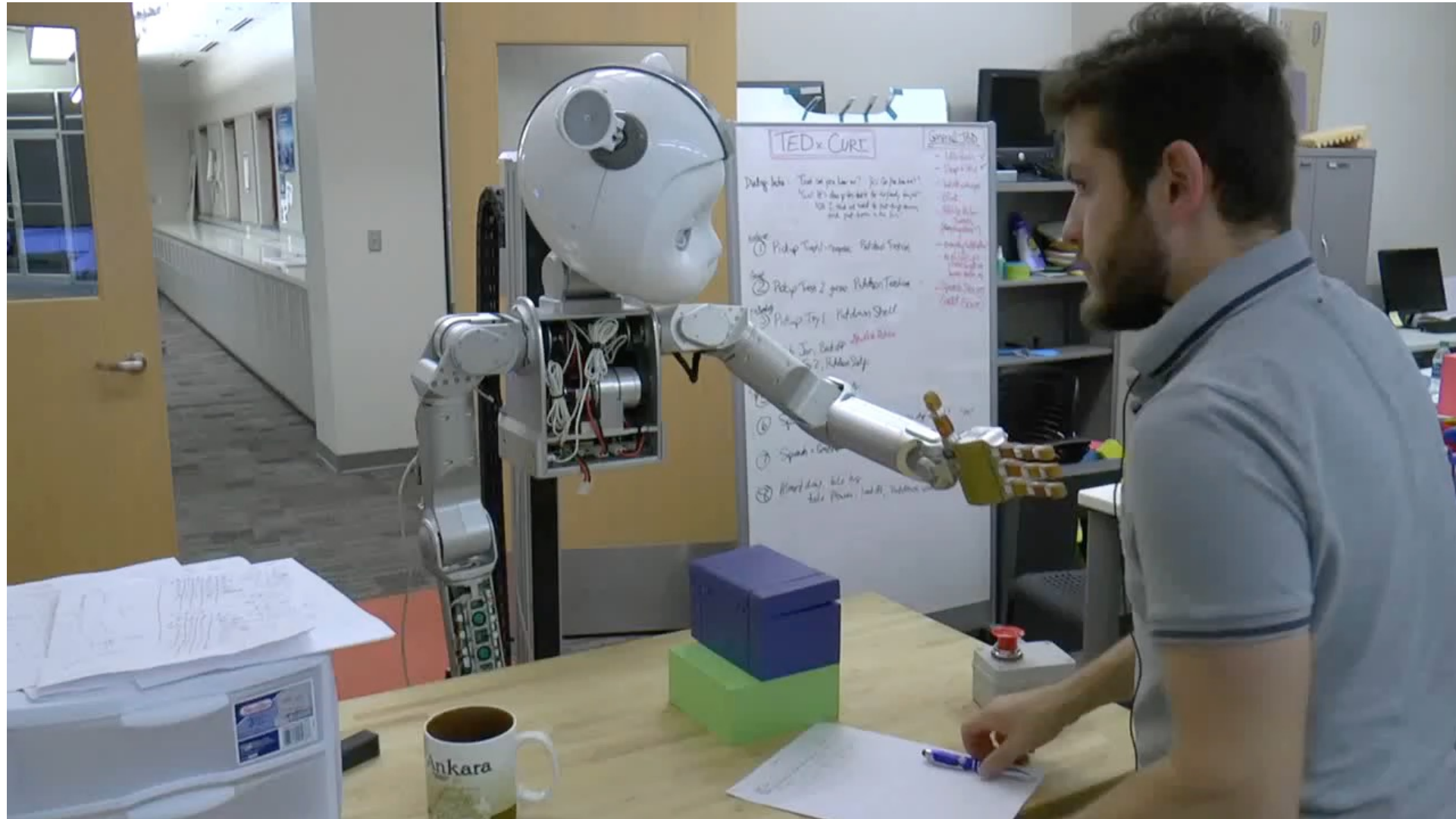
These descriptions lack several fundamental points, but the most important one is computational tractability and efficiency

# Decision Making

---

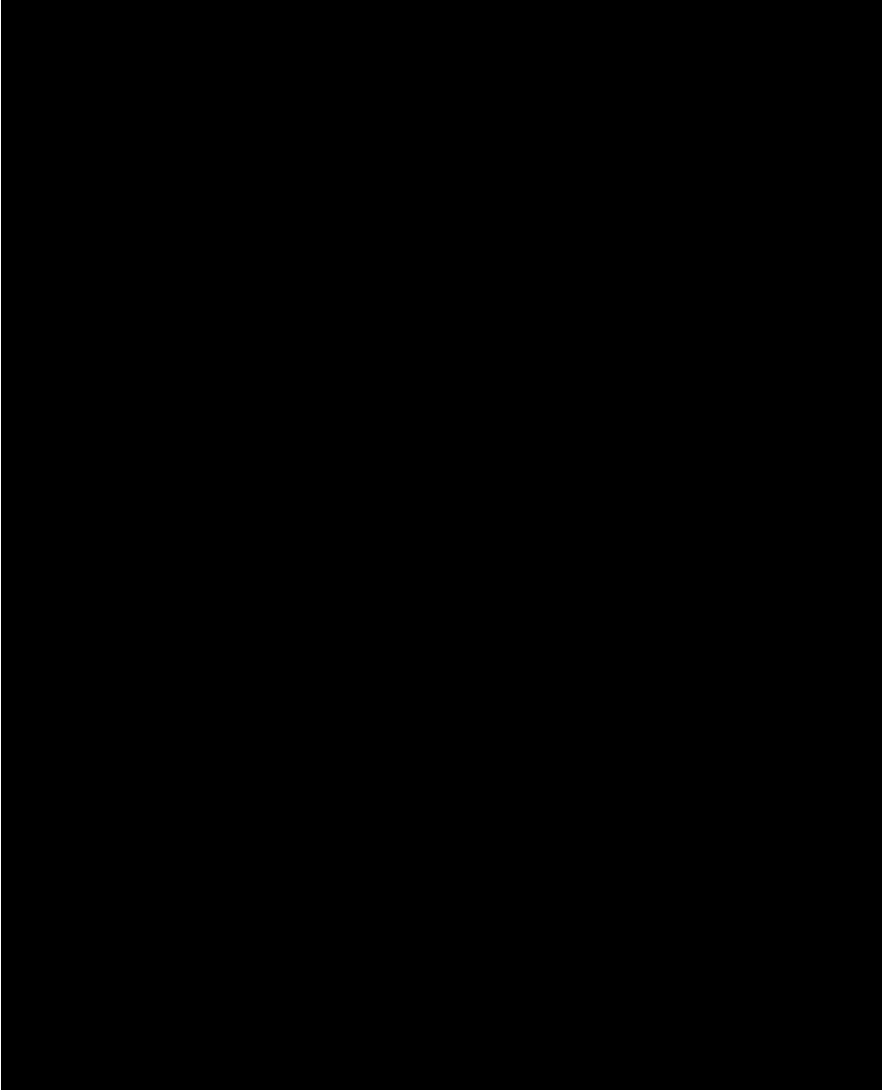
- What “**action**” should I take to maximize my “**expected**” outcome given the current “**situation**”?
- Any ideas...
- In real life:
  - Need to make sequential decisions
  - Outcomes of actions are uncertain
  - There is limited information
  - Rewards can also be uncertain (at least unknown before getting them)
- Reinforcement Learning: “Learning to make decisions by interacting with the environment”

# Robots!



# Famous Atari Example

---



In all of these examples, the agents were only given a reward function and time to interact with their environment!

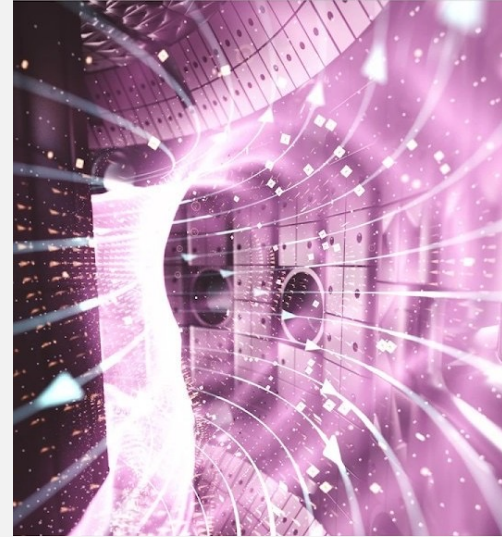
# Reinforcement Learning Examples: Why RL?



## Go — AlphaGo, 2016

Improving play through self-play

Image: Google DeepMind



## Fusion — DeepMind and EPFL, 2022

Learning to control plasma shape

Illustration: Google DeepMind



## Robot movement — ANYmal, 2019

Learning movement in simulation, then using it on a real robot

Image: Hwangbo et al., Science Robotics, 2019

Which response is better?

Response A

\_\_\_\_\_

✓ preferred

Response B

\_\_\_\_\_

\_\_\_\_\_

## Language — ChatGPT, 2022

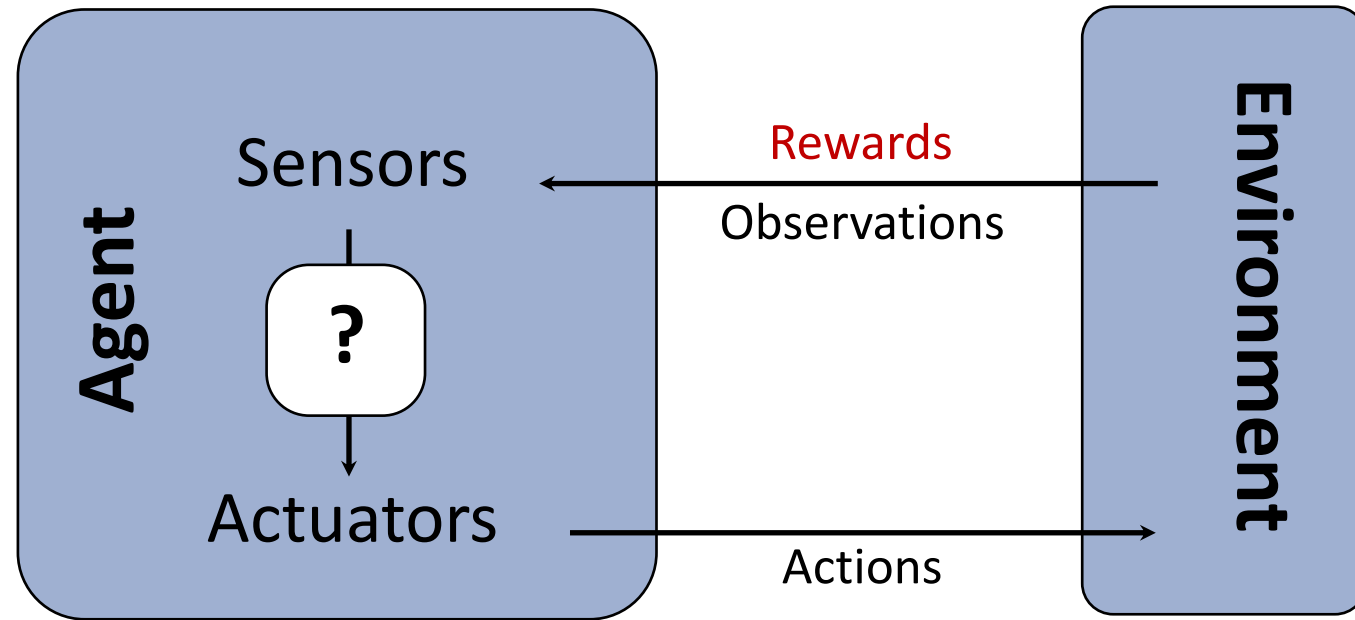
Improving responses with feedback learned from human comparisons

# How does RL differ from other learning methods?

| Learning setting              | Typical learning signal                      |
|-------------------------------|----------------------------------------------|
| <b>Supervised learning</b>    | Examples of the desired output               |
| <b>Unsupervised learning</b>  | Structure in the data, without target labels |
| <b>Reinforcement learning</b> | Rewards from the consequences of actions     |

In RL, actions affect later experience. A reward need not tell us which action was best.

# Agent and environment



The agent chooses an action. (Based on what?)

The environment responds with an observation and a reward.

A policy describes how the agent chooses actions.

# Concrete Example: Recycling Robot

---

- Mobile robot in an office
- Its task is to collect empty cans
- Solve it with RL
  - “robot”
  - “office”
  - “interaction”
  - “decisions”
  - “learn”

# Reward and return

---

- **Reward:** Feedback for each step (i.e. each time an action is taken).
- **Return:** Added up "future" rewards (later ones may count less).
  - Another interpretation: the rewards still to come
  - Can be defined for each time step!
- Recharging earns nothing immediately, but changes what can happen next. (0 reward yet it is important for success)

# Learning from experience

---

- If the robot is not given the consequences of its actions, how could it learn them?
- On a trial, it observes what happened for the action it chose, not directly what the alternatives would have produced.
- **Exploration:** trying actions to learn more.
- **Exploitation:** using what it has learned to obtain reward.

# Observation and state

---

- **Observation:** what the agent receives now (e.g. through its sensors)
- **State:** a description used to predict consequences and choose actions.
  - A useful state must retain the information that matters for what happens next and ideally not include irrelevant information, sort of an “abstraction”
  - Built from observations
- The battery gauge is hidden. You have just collected two cans.
- **Is the battery high or low?**
- **What earlier events could help you decide what to do next?**

# Formulating a decision problem

---

- For the recycling robot:
  - What decisions does it make, and what feedback does it receive?
  - Why might the latest observation be insufficient?
  - Why doesn't one good outcome settle which behaviour to use?

# Concrete Example: Recycling Robot

- Mobile robot in an office
- Its task is to collect empty cans

- Simplified Formulation:

- State/Observations
- How the “world works”
- Actions
- Rewards

- Example *transitions*:

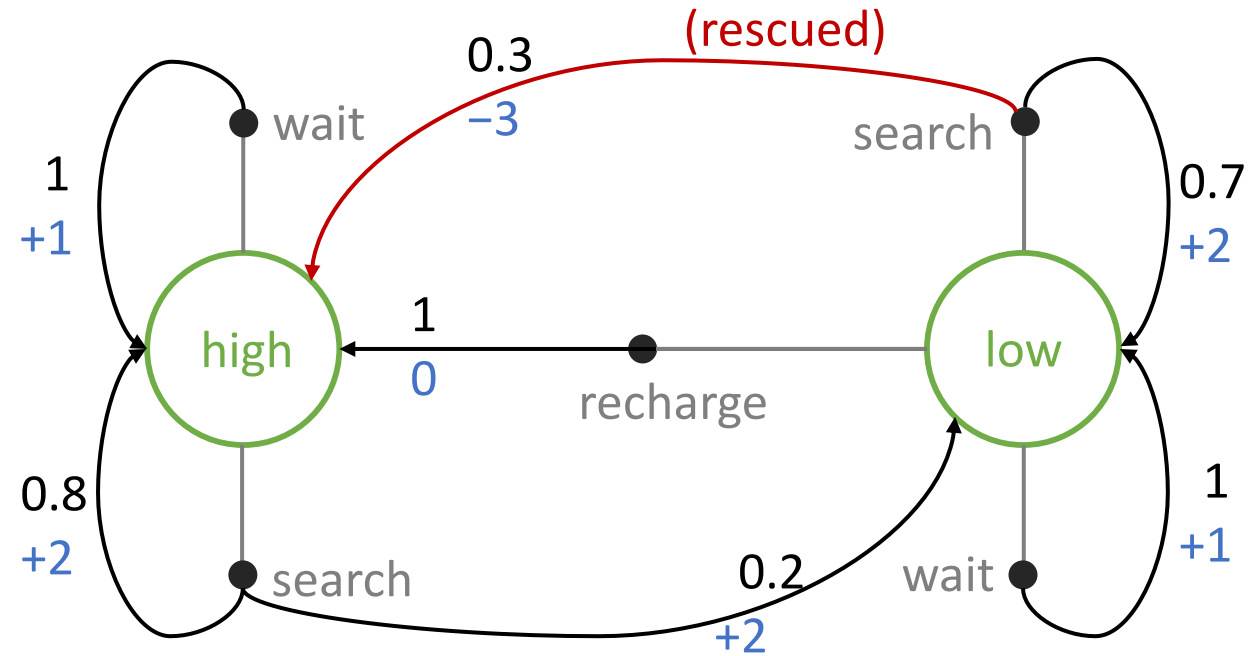
- Always search:

high, search, 2  $\rightarrow$  high, search, 2  $\rightarrow$  low, search, -3  $\rightarrow$  high, search, 2  $\rightarrow$  high, ...

- Search when high, recharge when low:

high, search, 2  $\rightarrow$  high, search, 2  $\rightarrow$  low, recharge, 0  $\rightarrow$  high, search, 2  $\rightarrow$  high, ...

- Returns?
- Side Question: Which “strategy” is better?



Adapted from Sutton & Barto, Example 3.3. Probabilities and search/wait rewards are illustrative.

# Summary: Recycling robot, simplified formulation

| Component                                    | Choice in this example                                                                                                                                                                                                     |
|----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Decisions                                    | Search, wait, recharge                                                                                                                                                                                                     |
| State (same as observation for this example) | High or low battery charge                                                                                                                                                                                                 |
| Model                                        | Search may lower the charge: from high it stays high with 0.8 (else low); from low it stays low with 0.7 (else the battery runs out and the robot is rescued to high). Wait keeps the charge; recharge returns it to high. |
| Reward                                       | Successful search: 2; wait: 1; recharge: 0; depletion/rescue: -3                                                                                                                                                           |
| Objective                                    | Collect cans over time while accounting for rescue costs                                                                                                                                                                   |
| Horizon                                      | Continuing                                                                                                                                                                                                                 |

# Why is RL different?

- Learns from **evaluative** feedback (rewards: how good was what I did?), not only **instructive** feedback (labels: what should I have done?)
  - and can also use a teacher when available, e.g. human game records (AlphaGo) or human comparisons (ChatGPT)
- Decisions are made sequentially (agent paradigm) – not i.i.d.!
- Agent's actions affect the subsequent data it receives
- Feedback is usually delayed (action now may have a huge impact in the future)
- **Next:** Define the model, the return objective and the policy.